



# Compilers: Principles, Techniques, and Tools

Min ZHANG

mzhang@sei.ecnu.edu.cn

# Lexcial Analysis

From RE to NFA & NFA to DFA

## 3.7 Converting A Regular Expression into A NFA (Thompson's Construction)



We now focus on transforming a Reg. Expr. to an NFA

This construction allows us to take:

- Regular Expressions (which describe tokens)
- To an NFA (to characterize language)
- To a DFA (which can be “computerized”)
  - Minizing DFA

The construction process is component-wise

Builds NFA from components of the regular expression in a special order with particular techniques.

**NOTE:** Construction is “syntax-directed” translation, i.e., syntax of regular expression is determining factor for NFA construction and structure.



# Thompson's Construction (cont.)

- ◆ This is one way to convert a regular expression into a NFA.
- ◆ There can be other ways (much efficient) for the conversion.
- ◆ Thompson's Construction is simple and systematic method.

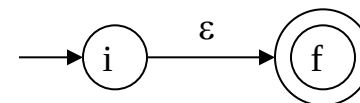
It guarantees that the resulting NFA will have exactly one final state, and one start state.

- ◆ Construction starts from simplest parts (alphabet symbols).

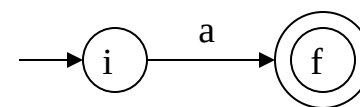
To create a NFA for a complex regular expression, NFAs of its sub-expressions are combined to create its NFA.

# Thompson's Construction (cont.)

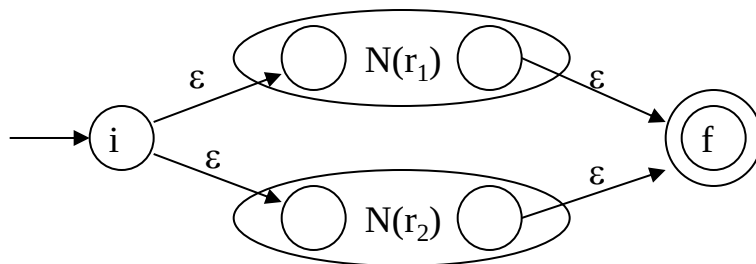
- ◆ To recognize an empty string  $\epsilon$



- ◆ To recognize a symbol  $a$  in the alphabet  $\Sigma$



- If  $N(r_1)$  and  $N(r_2)$  are NFAs for regular expressions  $r_1$  and  $r_2$ 
  - For regular expression  $r_1 \mid r_2$



NFA for  $r_1 \mid r_2$

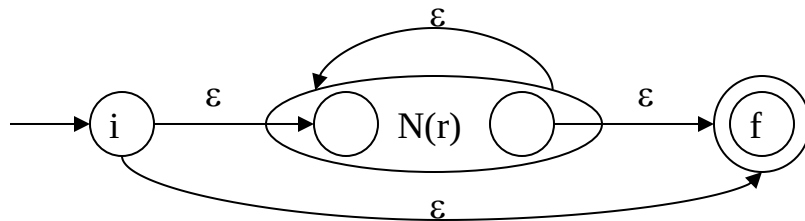
# Thompson's Construction (cont.)

- For regular expression  $r_1 r_2$



NFA for  $r_1 r_2$

- For regular expression  $r^*$



NFA for  $r^*$



# Construction Algorithm : R.E. $\rightarrow$ NFA

Construction Process :

1<sup>st</sup> : Identify subexpressions of the regular expression

$\epsilon$

$\Sigma$  symbols

$r \mid s$

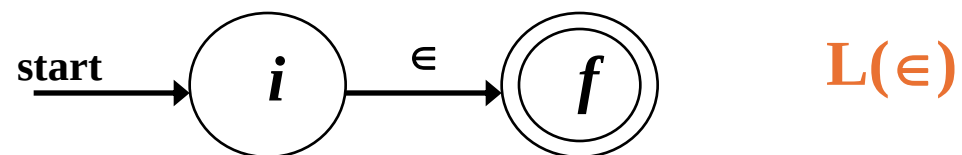
$rs$

$r^*$

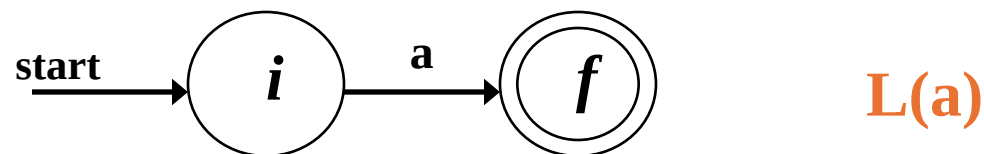
2<sup>nd</sup> : Characterize “pieces” of NFA for each subexpression

# Piecing Together NFAs

1. For  $\epsilon$  in the regular expression, construct NFA

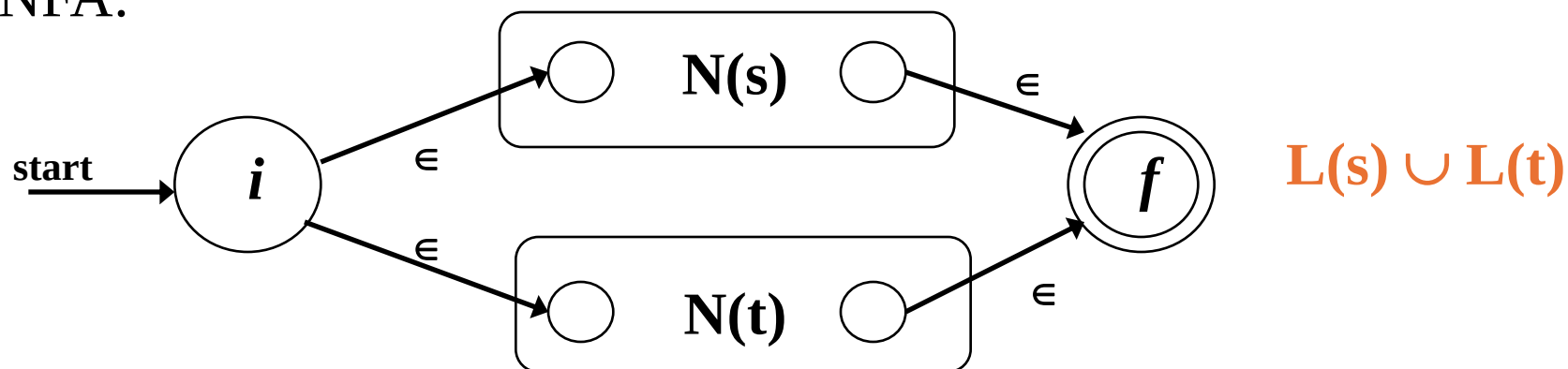


2. For  $a \in \Sigma$  in the regular expression, construct NFA



# Piecing Together NFAs – continued(1)

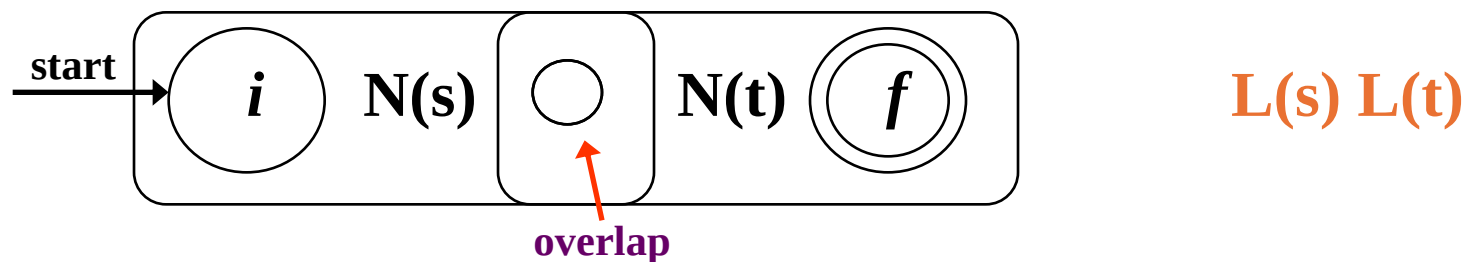
3.(a) If  $s$ ,  $t$  are regular expressions,  $N(s)$ ,  $N(t)$  their NFAs  $s|t$  has NFA:



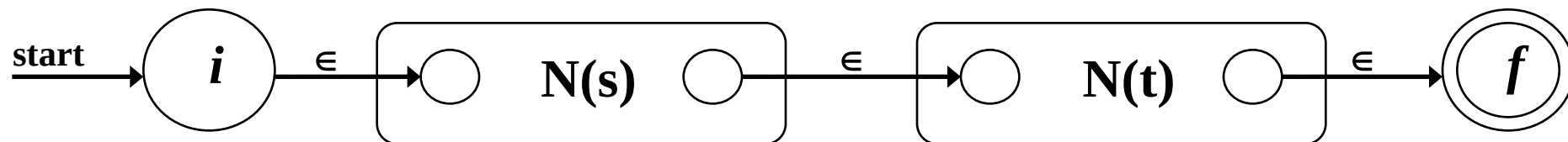
where  $i$  and  $f$  are new start / final states, and  $\epsilon$ -moves are introduced from  $i$  to the old start states of  $N(s)$  and  $N(t)$  as well as from all of their final states to  $f$ .

# Piecing Together NFAs – continued(2)

3.(b) If  $s, t$  are regular expressions,  $N(s), N(t)$  their NFAs  $st$  (concatenation) has NFA:



**Alternative:**

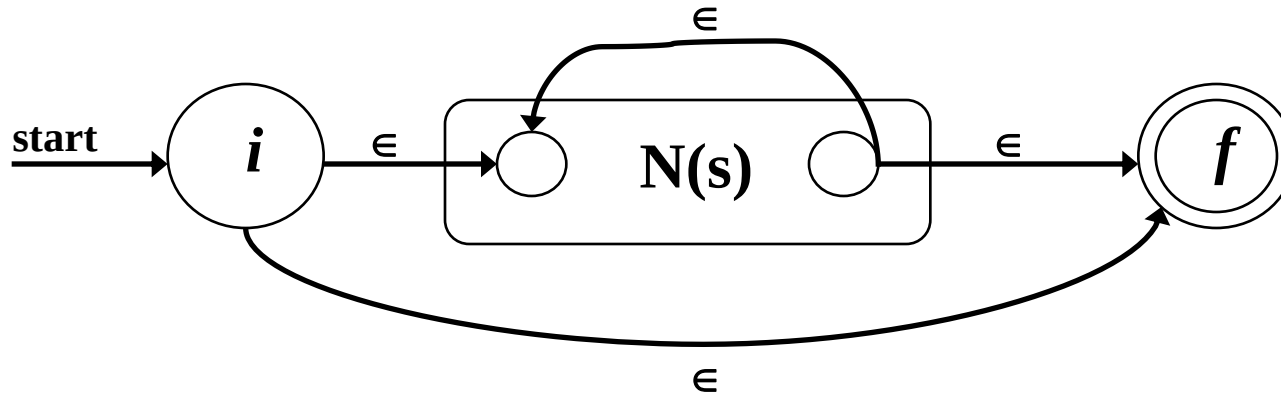


where  $i$  is the start state of  $N(s)$  (or new under the alternative) and  $f$  is the final state of  $N(t)$  (or new). Overlap maps final states of  $N(s)$  to start state of  $N(t)$ .

# Piecing Together NFAs – continued(3)



3.(c) If  $s$  is a regular expressions,  $N(s)$  its NFA,  $s^*$  (Kleene star) has NFA:



where :  $i$  is new start state and  $f$  is new final state

$\epsilon$ -move  $i$  to  $f$  (to accept null string)

$\epsilon$ -moves  $i$  to old start, old final( $s$ ) to  $f$

$\epsilon$ -move old final to old start (why?)

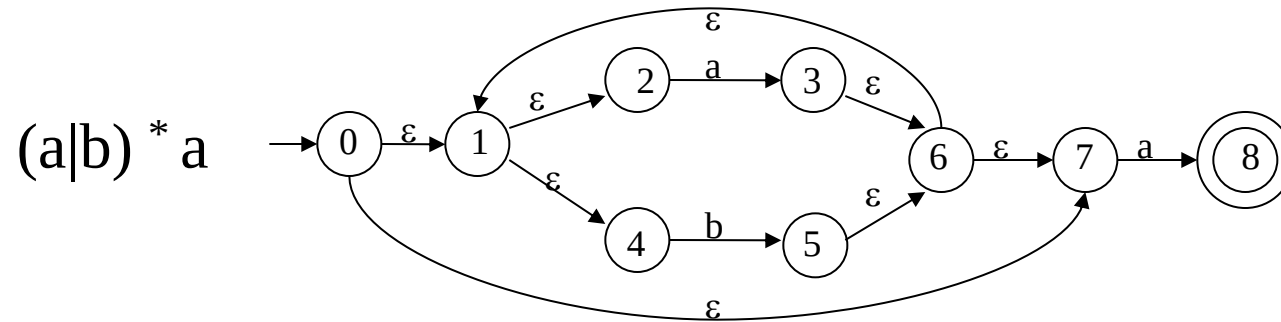
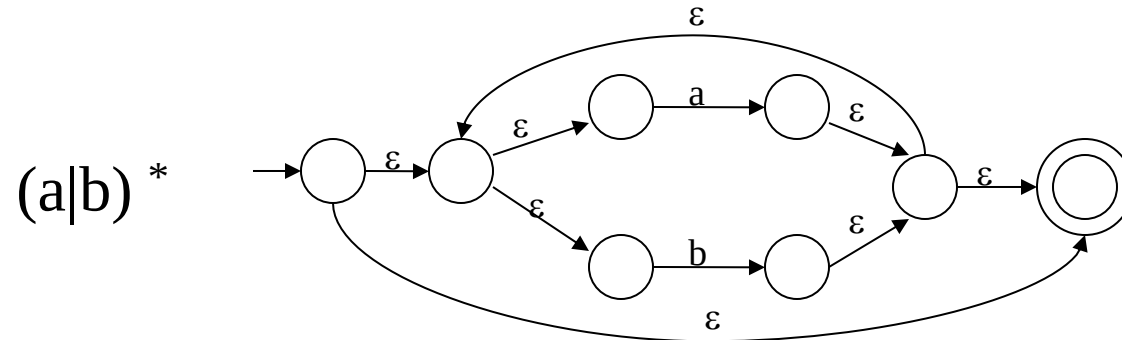
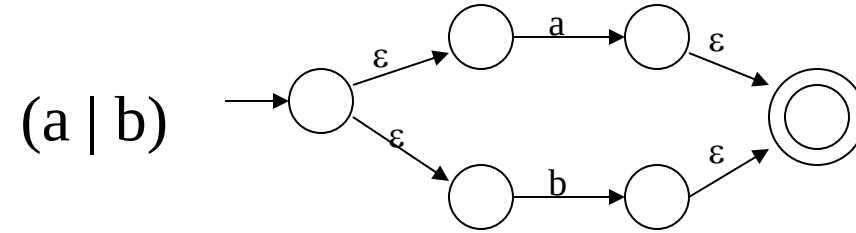
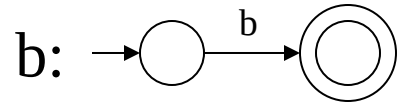
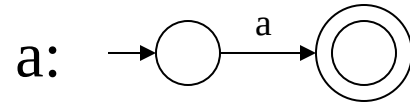


# Properties of Construction

Let  $r$  be a regular expression, with NFA  $N(r)$ , then

1.  $N(r)$  has #of states  $\leq 2 * (\text{\#symbols} + \text{\#operators})$  of  $r$
2.  $N(r)$  has exactly one start and one accepting state
3. Each state of  $N(r)$  has at most one outgoing edge  $a \in \Sigma$  or at most two outgoing  $\epsilon$ 's
4. **BE CAREFUL** to assign unique names to all states !

# Thompson's Construction Example 3.16 - $(a|b)^* a$



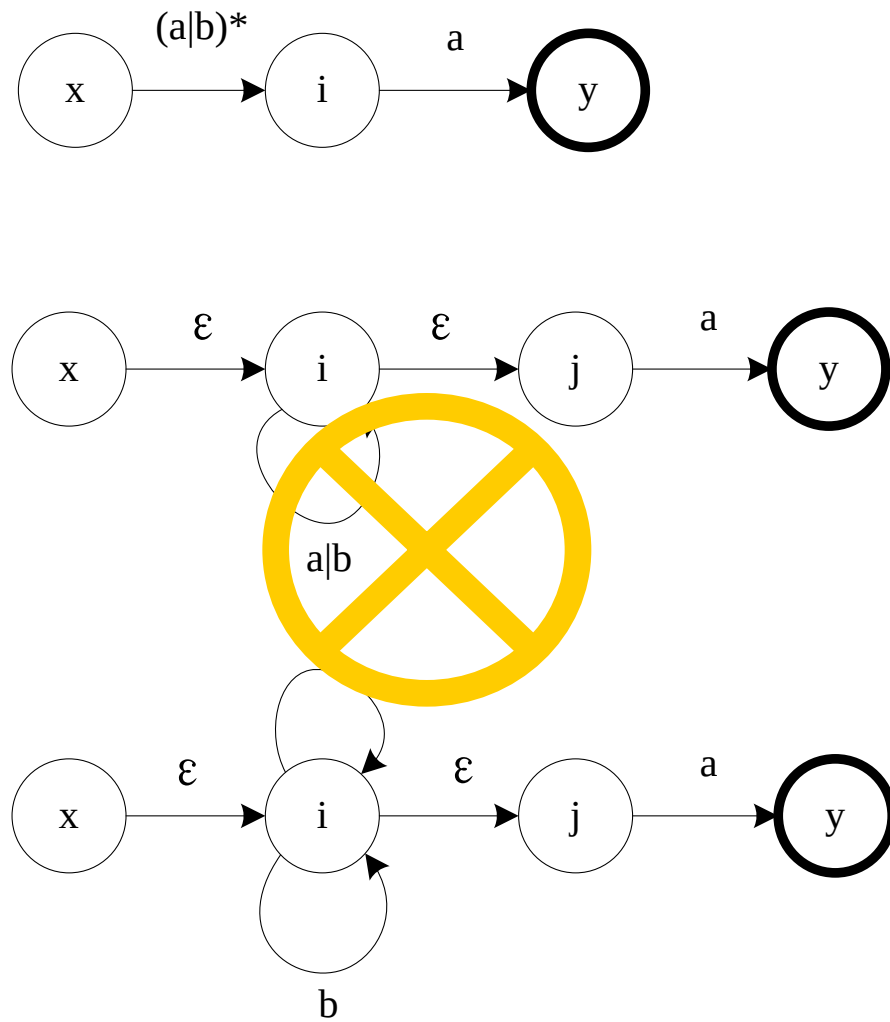


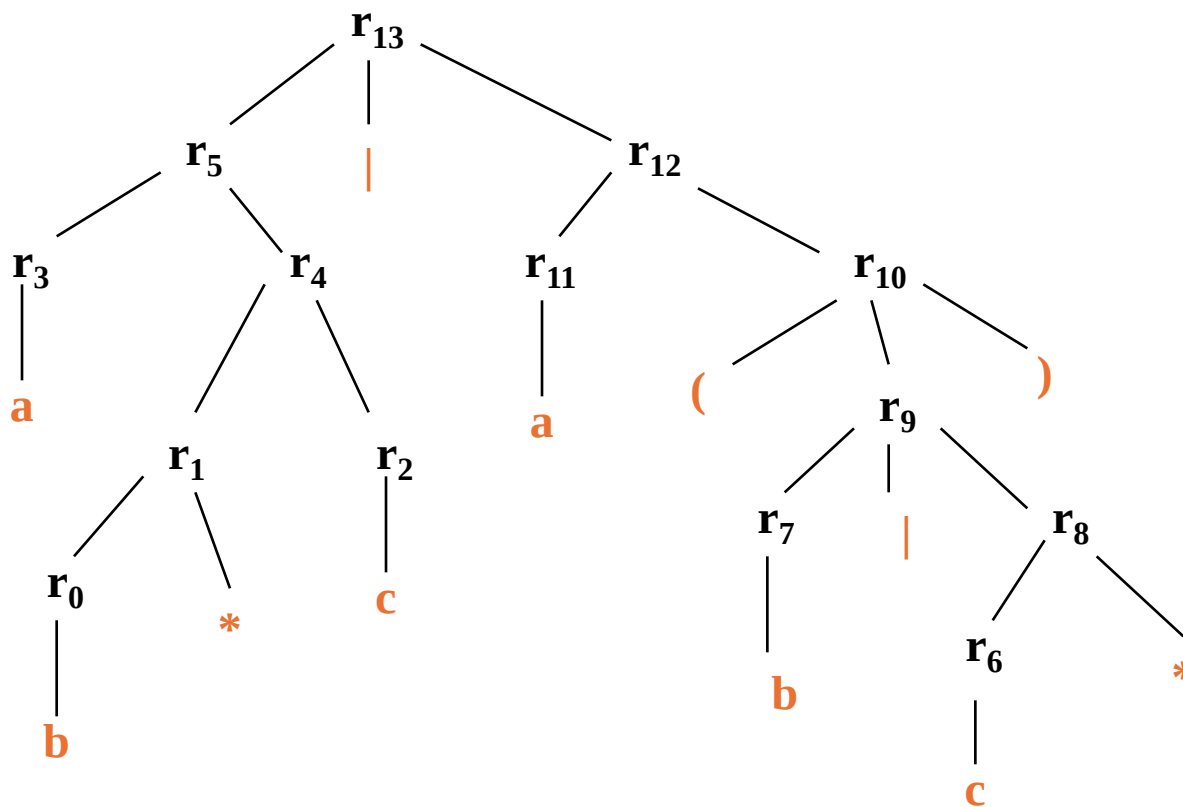
Fig. a. 3.1 another method to construct NFA  
from regular expression for Fig.3.p125



# R.E. $\rightarrow$ NFA (exercise)

2<sup>nd</sup> Example -  $(ab^*c) \mid (a(b \mid c^*))$

Parse Tree for this regular expression:

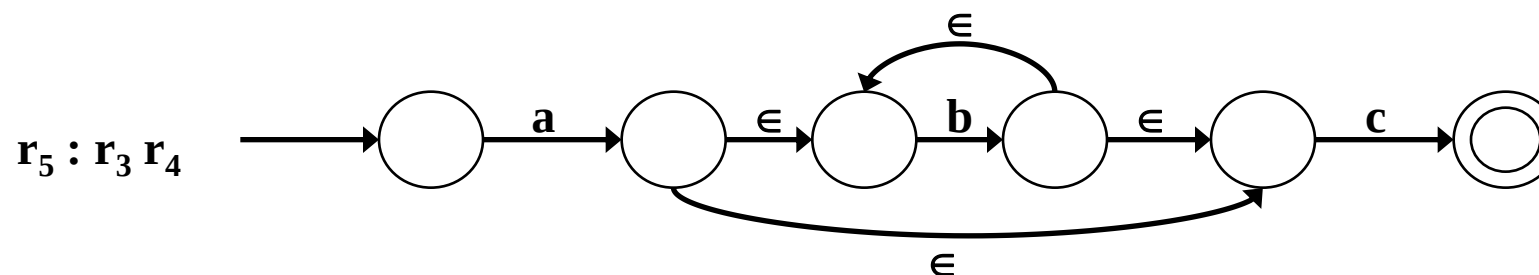
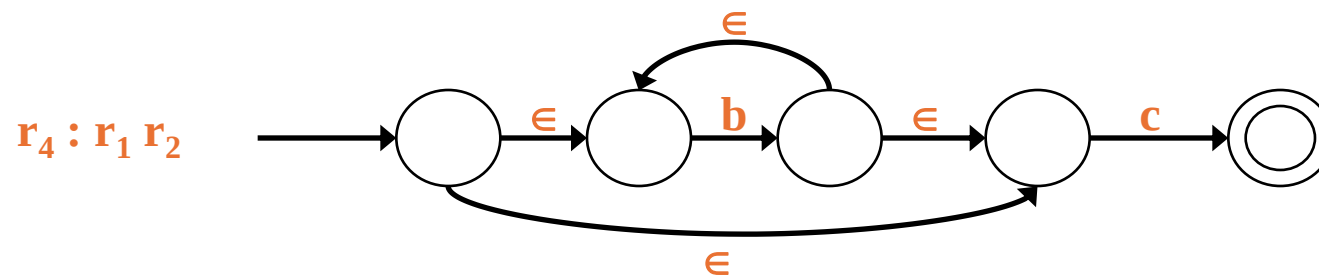
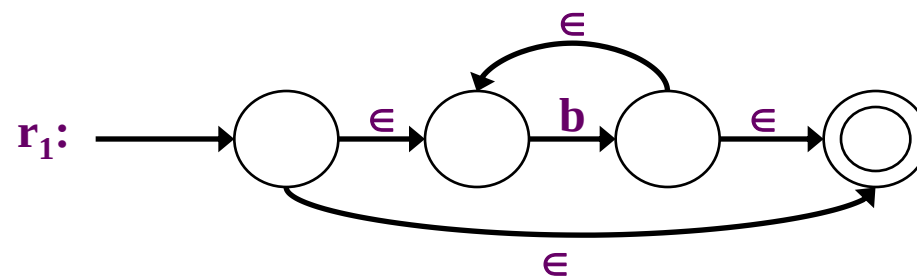
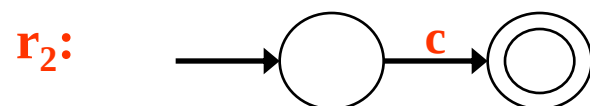
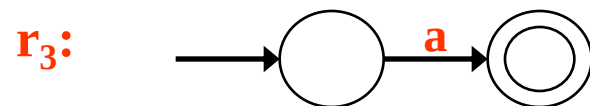
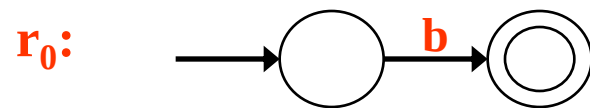


**What is the NFA? Let's construct it !**



# R.E. $\rightarrow$ NFA (exercise answer) – Construction(I)

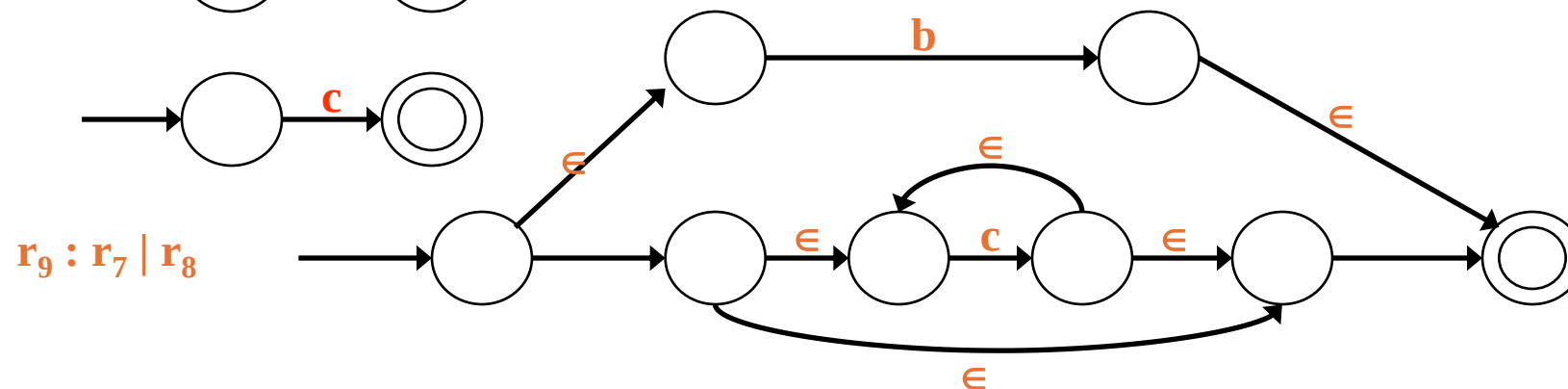
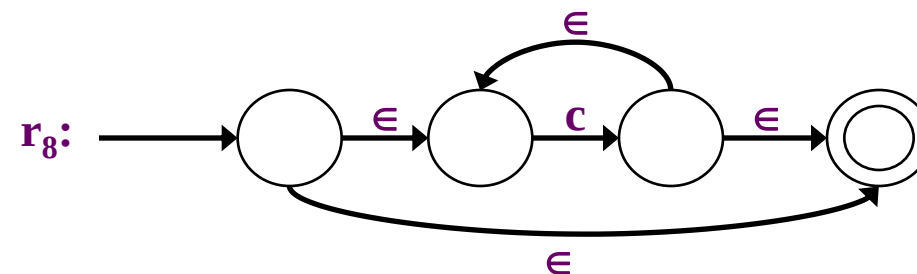
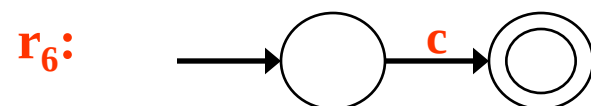
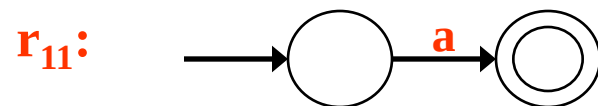
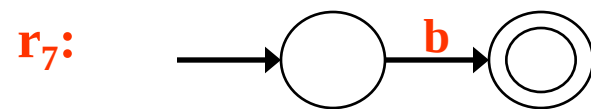
$(ab^*c) \mid (a(b|c^*))$



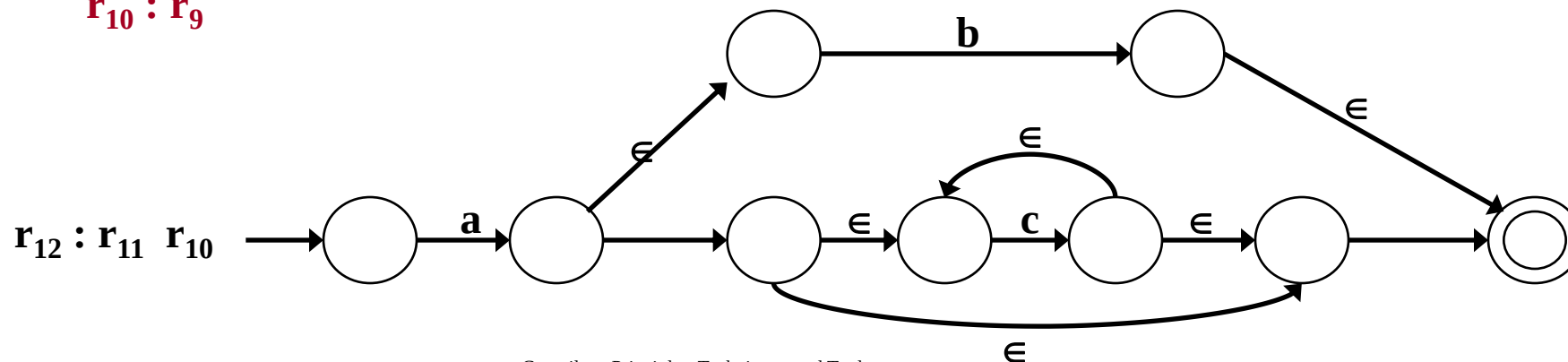


# R.E. $\rightarrow$ NFA (exercise answer) – Construction(2)

$(ab^*c) \mid (a(b|c^*))$



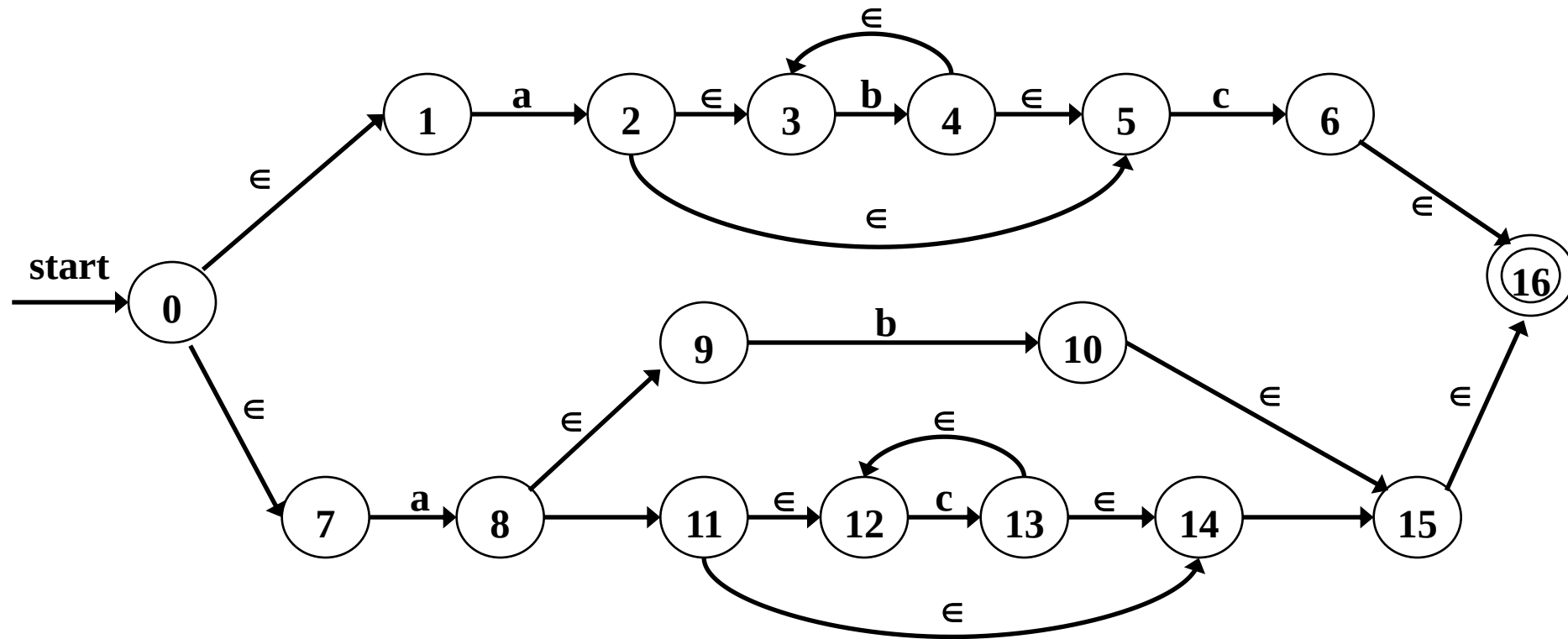
$r_{10} : r_9$



# R.E. $\rightarrow$ NFA (exercise answer) – Final Step



$r_{13} : r_5 \mid r_{12}$



# Direct Simulation of an NFA

```
s ← s0
c ← nextchar;
while c ≠ eof do
    s ← move(s,c);
    c ← nextchar;
end;
if s is in F then return "yes"
else return "no"
```

**DFA  
simulation**

Fig.3.31  
simulating  
the NFA of  
algorithm  
Thompson's  
construction

```
S ←  $\epsilon$ -closure({s0})
c ← nextchar;
while c ≠ eof do
    S ←  $\epsilon$ -closure(move(S,c));
    c ← nextchar;
end;
if  $S \cap F \neq \emptyset$  then return "yes"
else return "no"
```

**NFA  
simulation**

# Final Notes : R.E. to NFA Construction

- ◆ So, an NFA may be simulated by algorithm, when NFA is constructed using Previous techniques
- ◆ Algorithm run time is proportional to  $|N| * |x|$  where  $|N|$  is the number of states and  $|x|$  is the length of input
- ◆ Alternatively, we can construct DFA from NFA and use the resulting Dtran to recognize input:

|     | space<br>required | time to<br>simulate |
|-----|-------------------|---------------------|
| NFA | $O( r )$          | $O( r  *  x )$      |
| DFA | $O(2^{ r })$      | $O( x )$            |

where  $|r|$  is the length of the regular expression.

# Pulling Together Concepts

- Designing Lexical Analyzer Generator

Reg. Expr.  $\rightarrow$  NFA construction

NFA  $\rightarrow$  DFA conversion

DFA simulation for lexical analyzer

- Recall Lex Structure

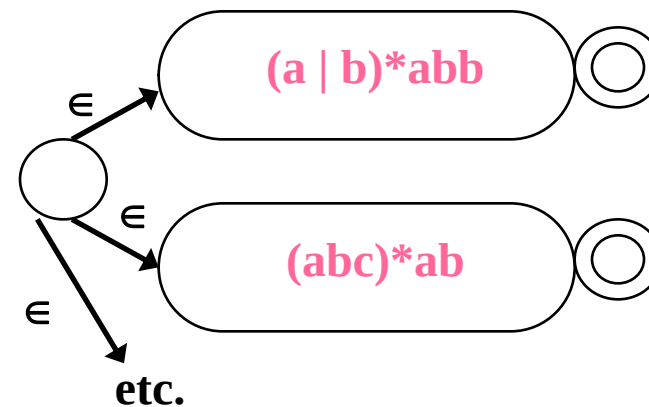
Pattern      Action

Pattern      Action

...

...

e.g.



- Each pattern recognizes lexemes

- Each pattern described by regular expression

**Recognizer!**



# Exercise in class

Using Thompson's construction to construct NFA from regular expression.

$$a(a|b)^+(b|c)(c|\epsilon)$$

# Converting a NFA into a DFA



- ◆ given an arbitrary NFA, construct an equivalent DFA (i.e., one that accepts precisely the same strings) need:

## 1、eliminating $\epsilon$ -transitions

an  $\epsilon$ -closure: the set of all states reachable by  $\epsilon$ -transitions from a state or states.

## 2、multiple transitions from a state on a single input character.

- ◆ keeping track of the set of states that are reachable by matching a single character.



- ◆ Both these processes lead us to consider **sets of states** instead of **single state**. Thus, it is not surprising that the DFA we construct has as its states *sets of states* of the original NFA.
- ◆ The algorithm is called the **subset construction** (子集构造法或子集法).

# Conversion : NFA $\rightarrow$ DFA



- Algorithm Constructs a Transition Table for DFA from NFA
- Each state in DFA corresponds to a **SET of states** of the NFA
- Why does this occur ?
  - $\epsilon$  moves
  - non-determinism

Both require us to characterize multiple situations that occur for accepting the same string.

(Recall : Same input can have multiple paths in NFA)

- Key Issue : Reconciling **AMBIGUITY** !

# Algorithm Concepts



| operation              | description                                                                                           |
|------------------------|-------------------------------------------------------------------------------------------------------|
| $\epsilon$ -closure(s) | Set of NFA states reachable from NFA state $s$ on $\epsilon$ -transitions alone.                      |
| $\epsilon$ -closure(T) | Set of NFA states reachable from some NFA states in $T$ on $\epsilon$ -transitions alone.             |
| Move(T,a)              | Set of NFA states to which there is a transition on input symbol $a$ from some NFA state $s$ in $T$ . |

Fig.3.24 Operations on NFA states.

# Algorithm Concepts (cont.)



NFA  $N = (S, \Sigma, s_0, F, \text{MOVE})$

$\epsilon\text{-Closure}(s) : s \in S$   
: set of states in  $S$  that are reachable from  $s$  via  $\epsilon$ -moves of  $N$  that originate from  $s$ .

No input is consumed

$\epsilon\text{-Closure}(T) : T \subseteq S$   
: NFA states reachable from all  $t \in T$  on  $\epsilon$ -moves only.

$\text{move}(T, a) : T \subseteq S, a \in \Sigma$   
: Set of states to which there is a transition on input  $a$  from some  $t \in T$

These 3 operations are utilized by algorithms / techniques to facilitate the conversion process.

# Algorithm for subset construction



```
put  $\epsilon$ -closure( $\{s_0\}$ ) as an unmarked state into the set of DFA (DStates)
while (there is one unmarked  $S_1$  in DStates) do
  begin
    mark  $S_1$ 
    for each input symbol  $a$  do
      begin
         $S_2 \leftarrow \epsilon$ -closure(move( $S_1, a$ ))
        if ( $S_2$  is not in DStates) then
          add  $S_2$  into DStates as an unmarked state
        transfunc[ $S_1, a$ ]  $\leftarrow S_2$ 
      end
    end
  end
```

$\epsilon$ -closure( $\{s_0\}$ ) is the set of all states  
can be accessible from  $s_0$  by  $\epsilon$ -  
transition.

set of states to which there is a  
transition on  $a$  from a state  $s$  in  $S_1$

Fig.3.25. the subset construction

- the start state of DFA is  $\epsilon$ -closure( $\{s_0\}$ )
- a state  $S$  in DStates is an accepting state of DFA if a state in  $S$  is an accepting state of NFA

# Algorithm For Subset Construction (cont.) (P.119)



```
push all states in T onto stack;  
initialize  $\epsilon$ -closure(T) to T;  
while stack is not empty do begin  
    pop t, the top element, off the stack;  
    for each state u with edge from t to u labeled  $\epsilon$  do  
        if u is not in  $\epsilon$ -closure(T) do begin  
            add u to  $\epsilon$ -closure(T) ;  
            push u onto stack  
        end  
    end  
end
```

computing the  $\epsilon$ -closure

Fig.3.26. computation of  $\epsilon$ -closure.

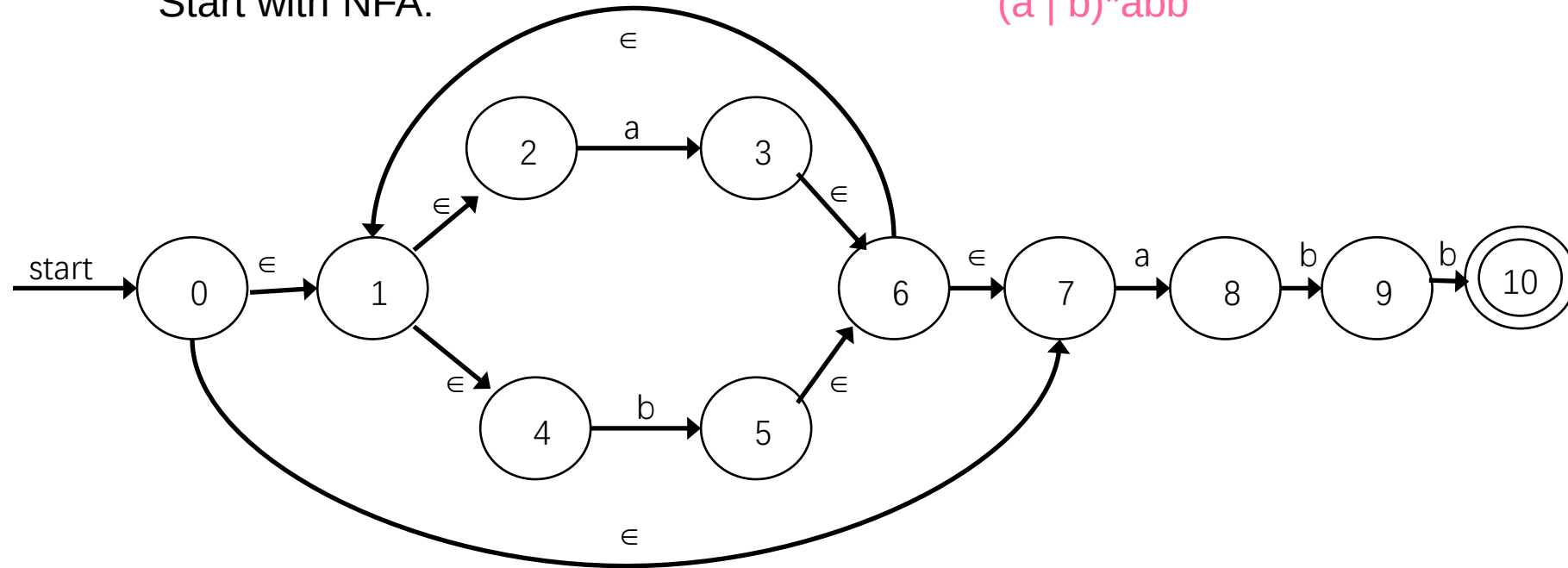
# Converting NFA to DFA – 1<sup>st</sup> Look

## (Example 3.15 P.119)



Start with NFA:

$(a \mid b)^*abb$



From State 0, Where can we move without consuming any input ?

This forms a new state: 0,1,2,4,7 What transitions are defined for this new state ?

# Converting a NFA into a DFA (calculate $\epsilon$ -closure )

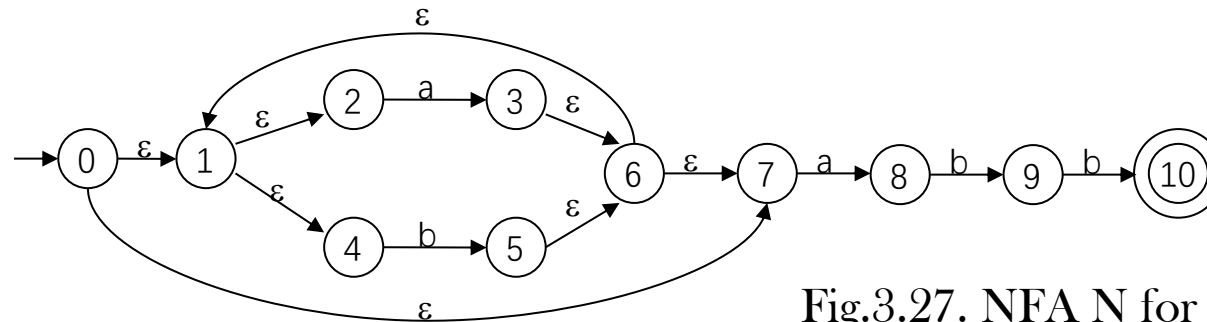


Fig.3.27. NFA N for  $(a|b)^*abb$

$\epsilon$ -closure( $\{0\}$ ) =  $\{0,1,2,4,7\} = S_0$   $S_0$  into DS as an unmarked state

$\Downarrow$  mark  $S_0$

$\epsilon$ -closure(move( $S_0, a$ )) =  $\epsilon$ -closure( $\{3,8\}$ ) =  $\{1,2,3,4,6,7,8\} = S_1$

$S_1$  into DS

$\epsilon$ -closure(move( $S_0, b$ )) =  $\epsilon$ -closure( $\{5\}$ ) =  $\{1,2,4,5,6,7\} = S_2$

$S_2$  into DS

transfunc[ $S_0, a$ ]  $\leftarrow S_1$

transfunc[ $S_0, b$ ]  $\leftarrow S_2$

$\Downarrow$  mark  $S_1$

$\epsilon$ -closure(move( $S_1, a$ )) =  $\epsilon$ -closure( $\{3,8\}$ ) =  $\{1,2,3,4,6,7,8\} = S_1$

$\epsilon$ -closure(move( $S_1, b$ )) =  $\epsilon$ -closure( $\{5,9\}$ ) =  $\{1,2,4,5,6,7,9\} = S_3$

transfunc[ $S_1, a$ ]  $\leftarrow S_1$

transfunc[ $S_1, b$ ]  $\leftarrow S_3$

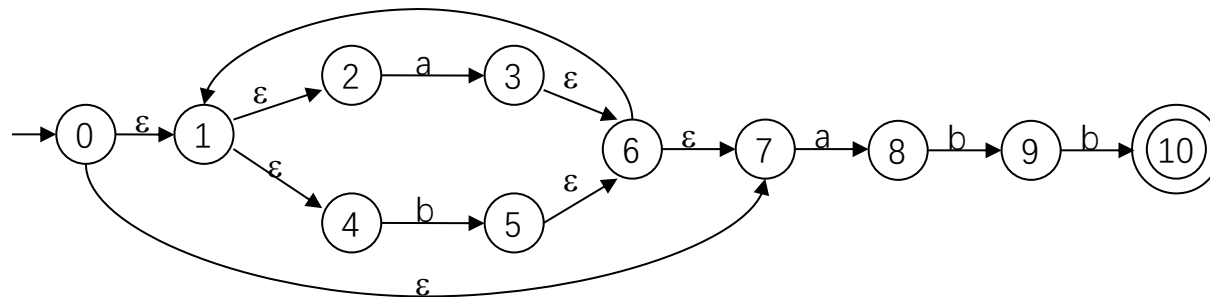
$\Downarrow$  mark  $S_2$

$\epsilon$ -closure(move( $S_2, a$ )) =  $\epsilon$ -closure( $\{3,8\}$ ) =  $\{1,2,3,4,6,7,8\} = S_1$

$\epsilon$ -closure(move( $S_2, b$ )) =  $\epsilon$ -closure( $\{5\}$ ) =  $\{1,2,4,5,6,7\} = S_2$

transfunc[ $S_2, a$ ]  $\leftarrow S_1$

transfunc[ $S_2, b$ ]  $\leftarrow S_2$



↓ mark S3

$\epsilon\text{-closure}(\text{move}(\text{S3}, a)) = \epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\} = \text{S1}$

$\epsilon\text{-closure}(\text{move}(\text{S3}, b)) = \epsilon\text{-closure}(\{5, 10\}) = \{1, 2, 4, 5, 6, 7, 10\} = \text{S4}$

$\text{transfunc}[\text{S3}, a] \leftarrow \text{S1}$

$\text{transfunc}[\text{S3}, b] \leftarrow \text{S4}$

↓ mark S4

$\epsilon\text{-closure}(\text{move}(\text{S4}, a)) = \epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\} = \text{S1}$

$\epsilon\text{-closure}(\text{move}(\text{S4}, b)) = \epsilon\text{-closure}(\{5\}) = \{1, 2, 4, 5, 6, 7\} = \text{S2}$

$\text{transfunc}[\text{S4}, a] \leftarrow \text{S1}$

$\text{transfunc}[\text{S4}, b] \leftarrow \text{S2}$

# Conversion Example – continued (4)



This gives the transition table **Dtran** for the DFA of:

| Dstates | Input Symbol |    |
|---------|--------------|----|
|         | a            | b  |
| S0      | S1           | S2 |
| S1      | S1           | S3 |
| S2      | S1           | S2 |
| S3      | S1           | S4 |
| S4      | S1           | S2 |

Fig.3.28. transition table **Dtran** for DFA.

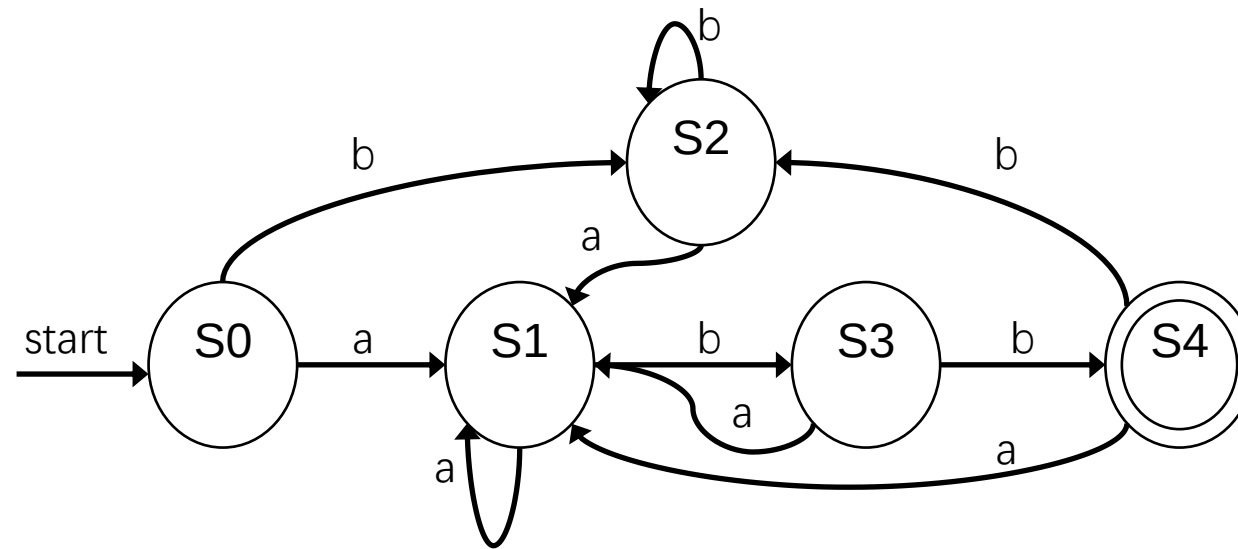


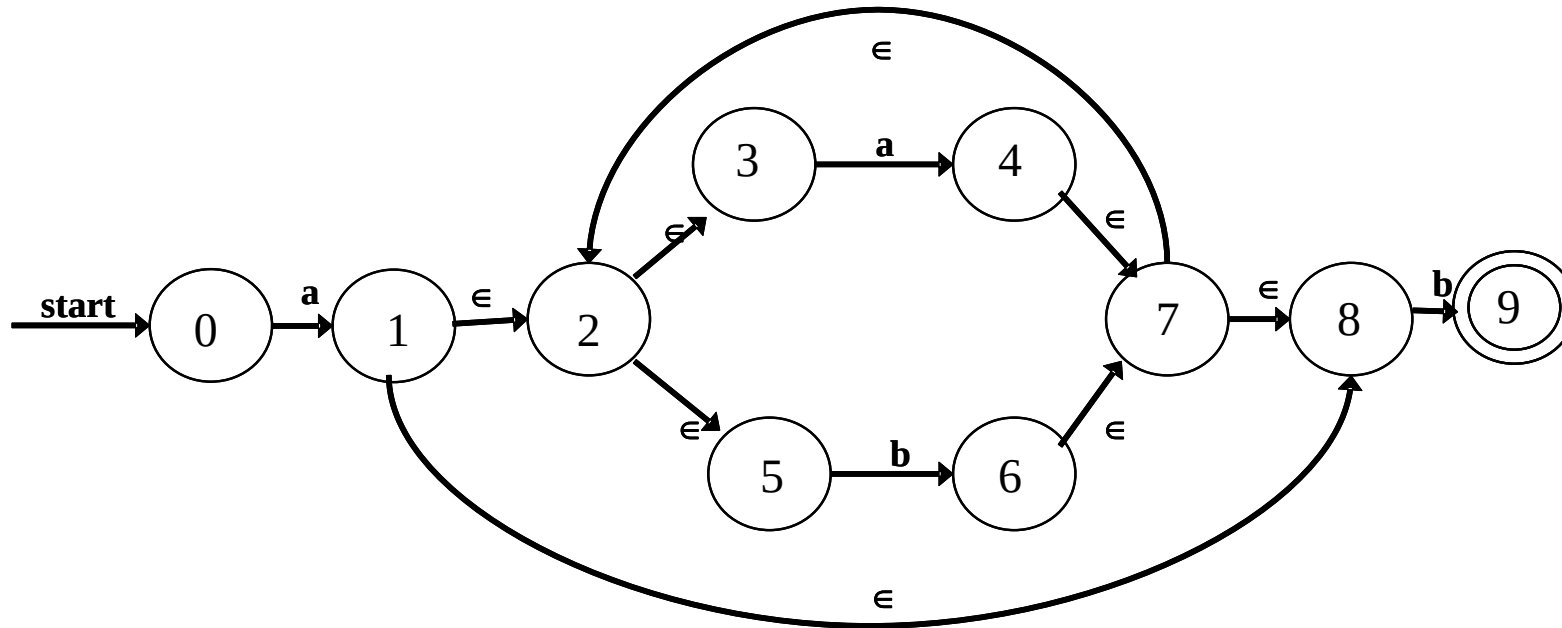
Fig.3.29. result of applying the subset construction to Fig.3.27. (P.121)

Fig.3.29. vs. Fig.3.23. (P.117)

# DFA (exercise in class)



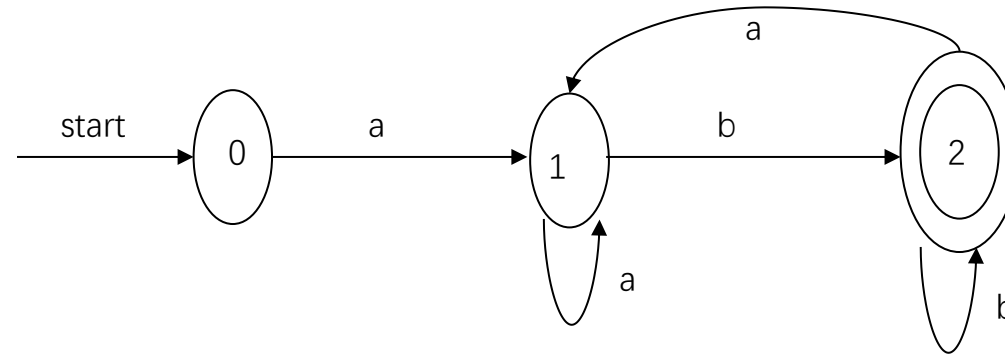
The language recognized by this DFA is also  $a(a|b)^*b$ , figure out a transition graph of DFA.



# DFA (exercise answer)



| Dstates | Input Symbol |    |
|---------|--------------|----|
|         | a            | b  |
| S0      | S1           |    |
| S1      | S1           | S2 |
| S2      | S1           | S2 |



DFA accepting  $a(a|b)^*b$

# Another exercise



- ◆ Using subset construction to construct DFA from NFA.

$$b(a|b)^+a$$

